

Assignment 2

[!IMPORTANT] This is not a class activity assignment (even though we started to work on this during class). This means that to get full credit (3 points) you must achieve the four criteria: **Correctness, Completeness, Punctuality** and **Complete Documentation**.



This assignment is due on: **October 7th**

Class activities are more flexible, you will get credit as long as it reflects true effort and there is no penalty for late submission.

Introduction

Comparative genomics

Comparative genomics asks a "simple question": **what do different genomes have in common, and what makes each one unique?**

We are approaching step by step to develop a big comparative genomics project, but before handling 20,000 proteins, let's practice with just five genes. Even though this is a small number of genes, you are starting to make a comparative analysis.

For this assignment, we will work with an **annotation file**. You already know the GTF format from our chromosome 21 and *E. coli* exercises. Now you will use it to compare **five genes** across **four species of vertebrates** separated by up to ~450 million years of evolution.

How this project works

You will do everything **manually first**: grep, awk and BLAST on the website. It will feel slow and repetitive, and that is on purpose. Then you will write the workflow **without code**, and finally you will write the **script** (your executable .sh file) that does all of it for you. We will run that script for real at Tule server in a couple of weeks.

Assignment Goals:

1. Download and explore real genome annotations from four species.
2. Extract gene information (coordinates, strand, transcripts, exons, protein accessions) using only **grep** and **awk**.
3. Use loops and conditionals to download many genes and species at once.
4. Check if the genes exist or not.
5. Translate a manual workflow into a scalable, reproducible script.

Our species and our genes

We will work with four species that are reference models.

Short name	Species	Genome assembly (NCBI RefSeq)
 mouse	<i>Mus musculus</i>	GRCm39 (GCF_000001635.27)

Short name	Species	Genome assembly (NCBI RefSeq)
 chicken	<i>Gallus gallus</i>	GRCg7b (GCF_016699485.2)
 frog	<i>Xenopus tropicalis</i>	UCB_Xtro_10.0 (GCF_000004195.4)
 zebrafish	<i>Danio rerio</i>	GRCz12tu (GCF_049306965.1)

And we will analyze five genes:

Gene	Biological function
TP53	Tumor suppressor, the "guardian of the genome". Stops cells with damaged DNA from dividing.
AIM2	Senses foreign DNA floating in the cytoplasm (e.g. from a virus) and triggers inflammation.
TLR9	Toll-like receptor that recognizes bacterial and viral DNA (CpG motifs) inside the cell.
TLR21	Another Toll-like receptor that also recognizes CpG DNA.
GULO	L-gulonolactone oxidase: the last enzyme needed to make vitamin C.



Exercise 1: Set up your project directories

Create your assignment directory named `assignment_2` inside your `btec_640/` directory with `input_data`, `final_output`, `analysis` and `src` inside.



Friendly reminder to always check that you are inside your working directory (`btec_640`) with `pwd`.

Inside `analysis/`, create three sub-directories: `gene_survey`, `proteins`, `blast`. Use a single command line.

►  Click here for a hint

#Paste your command here:

```
cd Documents/btec_640
pwd
ls
mkdir -p assignment_2
cd assignment_2
pwd
ls
mkdir -p input_data
mkdir -p final_output
mkdir -p analysis
mkdir -p src
ls
cd analysis
pwd
ls
```

```
mkdir -p {gene_survey, proteins, blast}
ls
```

Create a **README** file inside **assignment_2/** with a short description of this project, like you did in your first assignment.

Your structure should look like this:

 assignment_2

-  README
-  input_data
-  analysis
 -  gene_survey
 -  proteins
 -  blast
-  final_output
-  src

 **Remember** to document every command you use today in a separate script document, following our best practices.

Exercise 2: Download the four annotation files

Move inside **input_data/** and confirm that you are there? Paste the commands and the outputs:

#Paste your command and output here:

```
bash-3.2$ pwd
/Users/nchrste/Documents/btec_640/assignment_2
bash-3.2$ cd input_data/
bash-3.2$ pwd
/Users/nchrste/Documents/btec_640/assignment_2/input_data
```

Download the four GTF files from NCBI with **curl**:

```
curl -o mouse.gtf.gz
"https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/000/001/635/GCF_000001635.27_
GRCm39/GCF_000001635.27_GRCm39_genomic.gtf.gz"

curl -o chicken.gtf.gz
"https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/016/699/485/GCF_016699485.2_b
```

```
GalGal1.mat.broiler.GRCg7b/GCF_016699485.2_bGalGal1.mat.broiler.GRCg7b_genomic.gtf.gz"
```

```
curl -o frog.gtf.gz  
"https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/000/004/195/GCF_000004195.4_UCB_Xtro_10.0/GCF_000004195.4_UCB_Xtro_10.0_genomic.gtf.gz"
```

```
curl -o zebrafish.gtf.gz  
"https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/049/306/965/GCF_049306965.1_GRCz12tu/GCF_049306965.1_GRCz12tu_genomic.gtf.gz"
```

? Before continuing, do a sanity check: list the files with their sizes (`ls -lh`). Each file should be between ~15 and ~40 MB. What would it mean if one of them was only a few KB?

Type your answer:

If one of the files was only a few KB, then only part of the data was downloaded and not the complete data. Another option is the file is edited already to remove much of the information and was saved over the original input data. `geno`

Now unzip all four **using a for loop**, exactly like we did in the *E. coli* exercise.

[!CAUTION] Once unzipped, these four files together take **~3 GB** of disk space. Check that you have space before unzipping. Be careful with the `*` wildcard: make sure you are inside `input_data/`.

#Paste your loop here:

```
for GENOME in *gz;  
do gunzip $GENOME;  
done
```



Exercise 3: Explore the files

Go to `analysis/gene_survey/` and create a soft link to each GTF file, like we did with chromosome 21.

#Paste your command here:

```
cd ../  
pwd  
cd analysis_data/gene_survey  
pwd
```

```
ls
ln -s ../input_data/chicken.gtf analysis/gene_survey/
ln -s ../input_data/frog.gtf analysis/gene_survey/
ln -s ../input_data/mouse.gtf analysis/gene_survey/
ln -s ../input_data/zebrafish.gtf analysis/gene_survey/
ls
```

Open `mouse.gtf` with `less`, and then look at the first 10 lines with `head`.

? The first lines start with `#!`. What information do they contain? Why is this information important for reproducibility?

Type your answer:

The first lines contain the gene's name, accession number, and annotation data from NCBI. The first lines tell the version and identity of the gene. This is important for reproducibility because if someone else tries to use the same code but the version of the data is different, they might have different divisions of the raw data that would result in very different results of the same code.

? Compare column 1 of this file with column 1 of the `hg38.ncbiRefSeq.gtf` file from our chromosome 21 class. What is different? Using the RefSeq prefix table from that class, what does the `NC_` prefix tell you?

Type your answer:

Column 1 `hg38.ncbiRefSeq.gtf` is the chromosome ID while the first column in the `mouse.gtf` is the `NC_` reference number. `NC_` tells the complete genomic molecule.



Exercise 4: How many genes does each species have?

Each gene has exactly **one** line where column 3 is `gene`. Use `awk` to keep only those lines and count them with `wc -l`.

[!TIP] **Syntax: awk with a condition on a column**

```
awk -F'\t' '$3=="gene"' FILENAME
```

- `$3=="gene"` is the CONDITION: only lines where column 3 is exactly `gene` are printed.
- Note the **double** equal sign `==`. It means "is equal to" (a comparison), not "assign".

```
#Paste your commands here (one per species):
awk -F'\t' '$3=="gene"' chicken.gtf > chicken_gene
wc -l chicken_gene
awk -F'\t' '$3=="gene"' frog.gtf > frog_gene
wc -l frog_gene
awk -F'\t' '$3=="gene"' mouse.gtf > mouse_gene
wc -l mouse_gene
awk -F'\t' '$3=="gene"' zebrafish.gtf > zebrafish_gene
wc -l zebrafish_gene
```



To fill a table in markdown, each `||` represents a column. Look at my notes in the table below for reference.

Species	Number of genes
mouse	50766
chicken	25638
frog	28938
zebrafish	49663



Why do you think the numbers are different? Is the species with the most genes the "most complex"? Is the number of genes a biological fact, or a result of how well the genome was annotated?

Type your answer:

The numbers are different because each species has a different number of genes in their genome each portion of the gene has its own entry. The most genes does not mean the most complex. Many gene's are the same NC_ number but have different start and stop locations. The number of genes is a result of how well the genome was annotated because each section of one gene is receiving its own count.



Exercise 5: Dissect one gene by hand

Let's take one gene in one species and extract everything we can about it. We will use **TLR9 in mouse**.

In the NCBI annotation, every line that belongs to a gene carries its name in column 9 as `gene_id "NAME";`

[!TIP] **Syntax: grep with special characters**

```
grep -i 'gene_id "tlr9";' mouse.gtf > tlr9_mouse.gtf
```

- The **single quotes** ' ' protect everything inside, including the double quotes ".
- Why include `gene_id "` and `";`? Because `grep -i "tlr9"` alone would also catch any gene that *contains* `tlr9` in its name. Try it and see what happens!
- `-i` makes the search **case-insensitive**. Why could that matter across species? Keep this in mind.

Run the command above and open `tlr9_mouse.gtf`. Then answer:

1. How many lines does this file have? Which command did you use?

```
less tlr9_mouse.gtf
```

This command resulted in 8 lines of information in this file.

2. Using the line where column 3 is "gene": on which chromosome/sequence is `Tlr9`, what are its start and end coordinates?

```
Tlr9 is on NC_000075.7
Start: 106099797
Stop: 106104075
```

3. How long is the gene (in bp)? Show your calculation.

```
106104075 - 106099797 = 4378 bp long
```

4. How many exons does this gene has and which are the start and end position of each exon?

This gene has 1 exon and the start position is 106099797 and the end position is 106099905.

[!TIP] To see only the exon lines use: `awk -F'\t' '$3=="exon"' tlr9_mouse.gtf`

 A gene can have more than one transcript (isoform), and **each transcript lists its own exons**, so you may see the same exons repeated with a different `transcript_id`. Report the exons of the **first** transcript only.



Exercise 6: Repeat for all five genes

Repeat Exercise 5 for **all five genes in mouse and in chicken**. That is 10 **grep** commands. Fill the table:

Gene	Species	Found? (Y/N)	Chromosome/sequence	Start	End
TP53	mouse	N			
AIM2	mouse	Y	NC_00067.7	173177105	173293606
TLR9	mouse	Y	NC_00075.7	106099797	106104075
TLR21	mouse	N			
GULO	mouse	Y	NC_000080.7	66224235	66246703
TP53	chicken	Y	NW_024096016.1	5925	24899
AIM2	chicken	N			
TLR9	chicken	N			
TLR21	chicken	Y	NC_052542.1		
308996	335580				
GULO	chicken	N			

? Imagine doing this for 20,000 genes in 50 species. Which part of what you just did was **exactly the same** every time, and which part **changed**?

Type your answer:

```
grep -i "gene_id" stayed the same and the gene name and species were the two variables that changed
```

? Using VARIABLES and/or loops, how would you optimize this task? No need to write the command, write the logic and workflow that YOU would follow.

Type your answer:

```
define the variables for "GENE_NAME" and "SPECIES" then write a loop to read every "GENE_NAME" in each "SPECIES"'s genome
```



Exercise 7: Survey everything with a loop

You already know the **for** loop. Now we need **two** loops, one inside the other: for each species, go through every gene.

[!TIP] **Syntax: nested **for** loops**

```
for SPECIES in mouse chicken frog zebrafish
do
  for GENE in tp53 aim2 tlr9 tlr21 gulo
  do
    COMMAND using $SPECIES and $GENE
  done
done
```

? In your own words, explain what this command is doing and the **logic** behind it.

Type your answer:

The for SPECIES loop is saying to look through each SPECIES's genome. The for GENE loop is reading each gene in each of the SPECIES'S genome. The for GENE loop will run under each SPECIES specified under the for SPECIES loop. The COMMAND will tell us what to do for each GENE found in each SPECIES.

When we grep a gene that isn't in the file, **grep** still creates the output file, but it is **empty**. This is where **if/else** comes in. Remember the syntax from our chromosome 21 class:

[!TIP] **Syntax: **if** / **else** with [-s FILE]**

```
if [ -s "$file" ]
then
  COMMANDS_IF_TRUE
else
  COMMANDS_IF_FALSE
fi
```

- [-s "\$file"] asks: "does this file exist **and** is it not empty?"
- The syntax of IF/ELSE always require spaces inside the brackets: [-s "\$file"], if your command fails because of syntax the most common reason is that you did not left a space: [-s "\$file"].

[!TIP] **Syntax: variables inside double quotes**

Our grep pattern has double quotes inside it (`gene_id "tlr9"`). To use a **variable** inside a pattern, the pattern must be in **double** quotes (single quotes don't expand variables), so the inner quotes need a backslash `\`:

```
grep -i "gene_id \"${GENE}\";" ${SPECIES}.gtf
```

The backslash tells bash: "this `"` is a real character to search for, not the end of my text".

Complete the blanks and run the loop inside `analysis/gene_survey/`:

```
for SPECIES in mouse chicken frog zebrafish
do
    for GENE in tp53 aim2 tlr9 tlr21 gulo
    do
        grep -i "gene_id \"${GENE}\";" ${SPECIES}.gtf >
        ${GENE}_${SPECIES}.gtf

        if [ ${GENE}_${SPECIES}.gtf ]
        then
            echo "${GENE} ${SPECIES} FOUND"
        else
            echo "${GENE} ${SPECIES} NOT_FOUND"
            rm ${GENE}_${SPECIES}.gtf
        fi
    done
done > gene_survey.txt
```

[!NOTE] The `> gene_survey.txt` after the last `done` saves **everything** the loop prints into one file.

? Why do we remove the file in the `else` block? What would happen later if we left it there? If you don't know the answer, run the command without removing the file and see what happens.

Type your answer:

A Bunch of empty files would be present and could cause many issues later on

Open `gene_survey.txt` and copy the results into the table:

Gene	mouse	chicken	frog	zebrafish
TP53	NOT_FOUND	FOUND	FOUND	FOUND
AIM2	FOUND	NOT_FOUND	NOT_FOUND	NOT_FOUND

Gene	mouse	chicken	frog	zebrafish
TLR9	FOUND	NOT_FOUND	FOUND	FOUND
TLR21	NOT_FOUND	FOUND	FOUND	FOUND
GULO	FOUND	NOT_FOUND	NOT_FOUND	NOT_FOUND



Exercise 8: Download the protein sequences

A **NOT_FOUND** means only one thing: **grep did not find that text in that file**. It does not (yet) mean the gene is missing from the organism. There are several possible explanations:

1. The gene has a **different official name** in that species.
2. The gene is annotated, but without an official name yet (a placeholder ID, often **LOC** + numbers).
3. The gene is not annotated at all, but it **is** in the genome (the annotation pipeline missed it).
4. The gene is truly **absent** from the genome.

To check this, we can do a Blast search:

For every gene that **is** annotated download the protein sequence.

1. Build a file named **protein_list.txt** with three columns: gene, species, protein accession. Use a similar approach to the one we used in the Chromosome 21 exercise but instead of searching for the NM_ prefix, look for **NP_** or **XP_** because we want the Protein gene accession number. It should look like this:

```
tlr9 mouse NP_XXXXXXXXX.X
tlr9 frog XP_XXXXXXXXX.X
...
```

[!TIP] **Syntax: `grep -o` (print only the match)**

The protein accession is inside the **protein_id** attribute (in the **CDS** lines), not in the same position as the transcript accession we used in chromosome 21. To get it:

```
grep -o 'protein_id "NP_[^"]*"' tlr9_mouse.gtf | sort -u
```

- **-o** prints **only** the part of the line that matches, not the whole line.
- **[^"]*** means "any characters that are not a quote".

⚠ Some genes (especially in chicken, frog and zebrafish) only have a **predicted** protein, with the **XP_** prefix. If you don't find an **NP_**, search for **XP_** instead (like **tlr9 frog** in the example above).

#Paste the commands you used:

#Check for NP_ protein IDs

```

for SPECIES in mouse chicken frog zebrafish
do
  for GENE in tp53 aim2 tlr9 tlr21 gulo
  do
    if [ -s ${GENE}_${SPECIES}.gtf ]
    then
      PROTEIN=$(grep -o 'protein_id "NP_[^"]*"'
${GENE}_${SPECIES}.gtf | head -n 1 | awk -F'"' '{print $2}')
      echo "${GENE} ${SPECIES} ${PROTEIN}"
    fi
  done
done > protein_list.txt

#Check for XP_ protein IDs
for SPECIES in mouse chicken frog zebrafish
do
  for GENE in tp53 aim2 tlr9 tlr21 gulo
  do
    if [ -s ${GENE}_${SPECIES}.gtf ]
    then
      PROTEIN=$(grep -o 'protein_id "XP_[^"]*"'
${GENE}_${SPECIES}.gtf | head -n 1 | awk -F'"' '{print $2}')
      echo "${GENE} ${SPECIES} ${PROTEIN}"
    fi
  done
done >> protein_list.txt

```

2. Move to [analysis/proteins/](#) and download all the protein sequences **in a single while read loop**, like the chromosome 21 exercise.

⚠ In chromosome 21 exercise, we downloaded nucleotide sequences with `db=nucore`, WE NEED TO CHANGE THE URL BECAUSE WE ARE LOOKING FOR PROTEINS NO NUCLEOTIDES:

```

while read -r gene species accession
do
  curl -o "${gene}_${species}.faa"
  "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
db=protein&id=${accession}&rettype=fasta&retmode=text"
  sleep 1
done < ../gene_survey/protein_list.txt

```

Why `sleep 1`? NCBI blocks users who send too many requests per second. Being polite to a public server is part of best practices.

```

#Paste your command loop here:

while read -r gene species accession
do
  curl -o "${gene}_${species}.faa"

```

```
"https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
db=protein&id=${accession}&rettype=fasta&retmode=text"
sleep 1
done < ../gene_survey/protein_list.txt
```

3. Check that every `.faa` file has one header and a sequence (`grep -c ">" *.faa`). Then combine all species for each gene into **one multi-FASTA file** per gene (e.g. `tlr9_all.faa`).

Example:

```
cat tlr9_*.faa > tlr9_all.faa
```

Explain what does this command do and run it for every gene.

Type your answer

cat takes everything that shares the same condition and compiles it all into one file



Exercise 9: Confirm with BLAST

We want to know if a gene exists in a **genome**, even if nobody annotated it.

? We have a protein sequence (the query) and we want to search a genome (DNA). Which BLAST program do we need: `blastn`, `blastp`, `blastx` or `tblastn`? And why would `blastp` against the protein database **not** answer our question?

►  Click here for a hint

Type your answer:

We need to do `tblastn` because we searching for nucleotides based on the protein sequence we already have. `blastp` would compare a protein query to a protein database.

? Why do we use the **protein** instead of the DNA sequence to search in a species that diverged hundreds of millions of years ago?

Type your answer:

The same protein can be encoded by varying DNA sequences since the many amino acids have multiple 3 nucleotide codes. Searching for a DNA sequence would not give all the results for the same functional proteins.

Go to [NCBI BLAST](#) and choose **tblastn**.

- **Query:** paste the protein sequence that you got (the content of your **.faa** file).
- **Database:** [RefSeq Genome Database \(refseq_genomes\)](#).
- **Organism:** type the species you are testing and select it from the list.
- Click **BLAST** and be patient ⌚.

Run one search for every **NOT_FOUND** in your survey table (Exercise 7). As query, use the protein you downloaded from a species where the gene **was** found. Record the **top hit** and decide if the gene is present or absent:

#	Gene	Query (protein file)	Search in...	% Identity	Query cover	E-value	Present or Absent?
1	TP53	tp53_chicken.faa	mouse	69.1%	56%	4e-67	PRESENT
2	AIM2	aim2_mouse.faa	chicken				ABSENT
3	AIM2	aim2_mouse.faa	frog				ABSENT
4	AIM2	aim2_mouse.faa	zebrafish				ABSENT
5	TLR9	tlr9_mouse.faa	chicken	34.97%	96	1e-172	PRESENT
6	TLR21	tlr21_chicken.faa	mouse	29.65%	93%	1e-99	PRESENT
7	TLR21	tlr21_chicken.faa	frog	43.11%	93%	0	PRESENT
8	GULO	gulo_mouse.faa	chicken	54%	75.95%	2e-133	PRESENT
9	GULO	gulo_mouse.faa	frog	70.23%	100%	0	PRESENT
10	GULO	gulo_mouse.faa	zebrafish	31.93%	27%	3e-10	PRESENT

[!IMPORTANT] **A hit is not the same as the gene.** Proteins belong to families that share domains, so a TLR9 query will also find *other* Toll-like receptors. A real gene gives a hit that covers **most of the protein** (high query cover) with a **very small E-value**. A hit that covers only a small piece of the protein is usually just a shared domain from a different gene. Use search #1 (TP53) as your example of what a real hit looks like.

? Now fill the **final** table, using all your evidence: **P** = present, **A** = absent.

Gene	mouse	chicken	frog	zebrafish
TP53	P	P	P	P
AIM2	P	A	A	A
TLR9	P	P	P	P
TLR21	P	P	P	P
GULO	P	P	P	P

? Compare this table with your grep survey (Exercise 7). How many conclusions would have been **wrong** if you had stopped at the annotation file?

Type your answer:

A majority of the conclusions from just the annotation file were wrong. TP53, TLR9, TLR21, and GULO all had multiple conclusions that required more indepth comparison to determine if they were present or not.



Exercise 10: Write the workflow (no code)

You have now done every step of this analysis by hand. Without writing any code, describe the full workflow. For each step, write: **what goes in, what comes out, and which command/tool does it.**

Step 1: Make Directories

Input: Make Directory

Output: Each Directory

Tool/command: `mkdir -p`

Step 2: Download the annotation files

Input: *.gtf.gz files

Output: unzipped .gtf files

Tool/command: `curl -o` and `gunzip`

Step 3: soft link the files into the analysis directory

Input: *.gtf files in the input directory

Output: .gtf files in analysis directory

Tool/command: `ln -s`

Step 4: Count the genes

Input: The .gtf files

Output: counts of the genes in each file

Tool/command: `wc` and `awk`

Step 5: Identify a single gene

```
Input: *.gtf
Output: GENE*.gtf
Tool/command: grep -i and awk
```

Step 6: Repeat for all desired genes

```
Input: *.gtf
Output: GENE*.gtf
Tool/command: grep -i
```

Step 7: Use a loop to determine if the gene is found in each species

```
Input: *.gtf
Output: GENE_SPECIES.gtf
Tool/command: for, for, grep, echo, echo
```

Step 8: Download and read the protein sequence

```
Input: *.gtf
Output: protein.txt and *.faa files
Tool/command: grep -o, while read, and cat
```

Step 9: Run blast

```
Input: *.faa
Output: nucleotide blast
Tool/command: tblastn
```

? Which steps did you repeat many times? Those are your **loops**. Which steps depended on a result (found / not found)? Those are your **if/else**. Mark them in your workflow.



Exercise 11: Write your script

Write a script named `gene_survey_pipeline.sh` inside `assignment_2/src/`. **You will not run it yet:** we will run it on the cluster. Write it as if you were handing it to a colleague who has never seen this project.

On the cluster we won't use the BLAST website. We will download each genome, build a BLAST database, and run `tblastn` locally, refer to Blast website manual and use the syntax below to write your script:

[!TIP] **Syntax:** `makeblastdb`

```
makeblastdb -in GENOME.fna -dbtype nucl -out DATABASE_NAME
```

[!TIP] **Syntax:** `tblastn`

```
tblastn -query PROTEIN.faa \
        -db DATABASE_NAME \
```

```
-evaluate 1e-5 \  
-max_target_seqs 5 \  
-outfmt "6 qseqid sseqid pident length qstart qend sstart send  
evaluate bitscore qcovs" \  
-out RESULTS.tsv
```

- `-outfmt 6` writes a tab-separated table (one hit per line) instead of a long text report. The words after `6` choose the columns. Which of these columns match what you recorded in Exercise 9?
- The `\` at the end of a line means "the command continues on the next line". It makes long commands readable.

Your script must include:

- ☐ A header: purpose, author, date, how to run it, input files, output files, tools and versions.
- ☐ Variables at the top: at least `WORKDIR`, a list of species, and a list of genes. **No path should be written twice.**
- ☐ Directory creation (input, analysis sub-directories, output).
- ☐ Download of the GTF files **and** the genome FASTA files (hint: the genome file is in the same NCBI folder as the GTF, ending in `_genomic.fna.gz`).
- ☐ The survey loop with `if/else` (Exercise 7).
- ☐ Protein download for the genes that were found (Exercise 8).
- ☐ BLAST database construction for each genome.
- ☐ `tblastn` for every gene that was **NOT_FOUND**, using the protein of a species where it was found as query.
- ☐ A comment before every block explaining **what** it does and **why**.

To get credit

1. Upload your answers (all parts) in PDF format to Canvas under **Assignment2_answers**.
2. Upload the script under to Canvas **Assignment_script**